

GraphLab — Algorithms & Features Explanation Guide

Project: GraphLab — Interactive Graph Theory Laboratory

Purpose: This document explains all the algorithms and features used in GraphLab in simple terms, suitable for presenting to a teacher.

Table of Contents

1. [Project Overview](#)
2. [Graph Traversal Algorithms](#)
3. BFS (Breadth-First Search)
4. DFS (Depth-First Search)
5. [Shortest Path Algorithms](#)
6. Dijkstra's Algorithm
7. Bellman-Ford Algorithm
8. BFS Pathfinding
9. DFS Pathfinding
10. [Minimum Spanning Tree](#)
11. Prim's Algorithm
12. [Eulerian & Hamiltonian Paths](#)
13. [Complement Graph](#)
14. [Bipartite Graph Verification](#)
15. [Havel-Hakimi Theorem \(Degree Sequences\)](#)
16. [Graph Isomorphism](#)
17. [Adjacency Matrix & Incidence Matrix](#)
18. [AI Tutor & Image-to-Graph](#)

1. Project Overview

GraphLab is a **web-based interactive graph theory tool** built with **React**, **TypeScript**, and **Vite**. It allows users to:

- **Draw graphs** on a canvas (add nodes, connect edges)
- **Run algorithms** step-by-step with visual animation
- **Analyze graph properties** (complement, bipartite check, degree sequences, isomorphism)
- **Generate matrices** (adjacency matrix, incidence matrix)
- **Use AI** to explain graph theory concepts

The project uses an **adjacency list** data structure to store graphs internally:

```
Map<nodeId, GraphEdge[]>
```

Each node has an ID, coordinates (x, y), and radius. Each edge stores source, destination, weight, and type (directed/undirected).

2. Graph Traversal Algorithms

2.1 BFS (Breadth-First Search)

What it does: Explores a graph level by level, visiting all neighbors of a node before moving deeper.

How it works: 1. Start from a source node, add it to a **Queue** 2. While the queue is not empty: - Remove the front node from the queue - Mark it as **visited** - Add all its **unvisited neighbors** to the back of the queue 3. This ensures nodes are visited in order of their distance from the source

Data Structure Used: Queue (FIFO — First In, First Out)

Time Complexity: $O(V + E)$, where V = vertices, E = edges

Example: Starting from node A in graph A—B—C—D:

```
Step 1: Visit A, Queue = [B]
Step 2: Visit B, Queue = [C]
Step 3: Visit C, Queue = [D]
Step 4: Visit D, Queue = []
```

2.2 DFS (Depth-First Search)

What it does: Explores a graph by going as deep as possible along each branch before backtracking.

How it works: 1. Start from a source node, push it onto a **Stack** 2. While the stack is not empty: - Pop the top node from the stack - If not visited, mark it as **visited** - Push all its **unvisited neighbors** onto the stack 3. This creates a deep exploration path

Data Structure Used: Stack (LIFO — Last In, First Out)

Time Complexity: $O(V + E)$

Example: Starting from node A in a graph A—B—C, A—D:

```
Step 1: Visit A, Stack = [D, B]
Step 2: Visit B, Stack = [D, C]
Step 3: Visit C, Stack = [D]
Step 4: Visit D, Stack = []
```

3. Shortest Path Algorithms

3.1 Dijkstra's Algorithm

What it does: Finds the shortest path from a source node to all other nodes in a **weighted graph** (with non-negative weights).

How it works: 1. Set distance of source node = 0, all others = ∞ (infinity) 2. Use a **Priority Queue** (min-heap) to always pick the node with the smallest known distance 3. For the picked node, check all its neighbors: - If $\text{distance}[\text{current}] + \text{edge_weight} < \text{distance}[\text{neighbor}]$: - Update $\text{distance}[\text{neighbor}] = \text{distance}[\text{current}] + \text{edge_weight}$ - Record the path (parent pointer) 4. Repeat until all nodes are processed

Key Formula: $d(v) = \min(d(v), d(u) + w(u, v))$

This is called the **relaxation step**.

Data Structure Used: Priority Queue / Min-Heap

Time Complexity: $O((V + E) \times \log V)$ with a binary heap

Example:

Graph: $A \xrightarrow{-3} B \xrightarrow{-2} C$, $A \xrightarrow{-7} C$

Start at A:

$d(A) = 0$

$d(B) = 3$ (via $A \rightarrow B$)

$d(C) = \min(7, 3+2) = 5$ (via $A \rightarrow B \rightarrow C$, not direct $A \rightarrow C$)

Shortest path A to C = $A \rightarrow B \rightarrow C$, distance = 5

[!IMPORTANT] Dijkstra's algorithm does NOT work with negative edge weights. For that, use Bellman-Ford.

3.1.1 Random Weighted Graph Generation for Dijkstra

GraphLab includes an automatic **Random Weighted Graph Generator** specifically calibrated for Dijkstra's algorithm: -

Guaranteed Connectivity: First generates a random spanning tree so all vertices are reachable. - **Positive Edge Weights:** Generates weights randomly distributed between 1 and 15 (strictly non-negative, fulfilling Dijkstra's prerequisites). -

Alternative Pathways: Adds random cross-edges (density ~60%) so the visualizer demonstrates priority-queue path relaxation and optimal route discovery. - **Instant 1-Click Access:** Available directly via the Dijkstra instruction banner and the Template Library.

3.2 Bellman-Ford Algorithm

What it does: Finds shortest paths from a source node, and **can handle negative edge weights**. Also detects **negative weight cycles**.

How it works: 1. Set distance of source = 0, all others = ∞ 2. Repeat $(V - 1)$ times: - For every edge (u, v) with weight w : - If $\text{distance}[u] + w < \text{distance}[v]$, update $\text{distance}[v]$ 3. Do one more pass: if any distance can still be reduced, then a **negative cycle** exists

Why V-1 iterations? The shortest path in a graph with V vertices can have at most $V-1$ edges.

Time Complexity: $O(V \times E)$

Negative Cycle Detection: If after $V-1$ iterations, we can still relax an edge, it means there's a cycle with negative total weight, making shortest paths undefined.

3.3 BFS Pathfinding

What it does: Finds the shortest path between two nodes in an **unweighted graph**.

How it works: Same as BFS traversal, but: - Stops when the **target node** is reached - Backtracks using parent pointers to reconstruct the path

Why it works for shortest paths: In an unweighted graph, BFS explores nodes in order of distance (1 hop, 2 hops, etc.), so the first time it reaches the target is guaranteed to be the shortest path.

3.4 DFS Pathfinding

What it does: Finds a **path** (not necessarily the shortest) between two nodes.

How it works: Same as DFS traversal, but stops when the target is found and reconstructs the path.

[!NOTE] DFS pathfinding does NOT guarantee the shortest path. Use BFS or Dijkstra for shortest paths.

4. Minimum Spanning Tree

4.1 Prim's Algorithm

What it does: Finds the **Minimum Spanning Tree (MST)** — a subset of edges that connects all vertices with the minimum total weight, without forming any cycle.

How it works: 1. Start with any node as the "tree" 2. Use a **Priority Queue** to find the minimum weight edge connecting the tree to a non-tree vertex 3. Add that edge and vertex to the tree 4. Repeat until all vertices are in the tree

Key Property: MST has exactly **$V - 1$** edges for V vertices.

Time Complexity: $O((V + E) \times \log V)$

Example:

Graph: A-1-B, B-3-C, A-2-C

MST picks edges: A-1-B, A-2-C (total weight = 3)

NOT: A-1-B, B-3-C (total weight = 4)

5. Eulerian & Hamiltonian Paths

5.1 Eulerian Path/Circuit

What it does: Determines if a graph has an **Eulerian path** (visits every EDGE exactly once) or an **Eulerian circuit** (starts and ends at the same vertex).

Conditions: - **Eulerian Circuit:** Every vertex must have an **even degree** - **Eulerian Path:** Exactly **two vertices** must have an **odd degree** (these are the start and end vertices) - The graph must be **connected**

Algorithm used: Hierholzer's algorithm — builds the path by following edges and merging sub-tours.

Famous Example: The **Königsberg Bridge Problem** (1736) — Euler proved it was impossible because all 4 vertices had odd degree.

5.2 Hamiltonian Path/Circuit

What it does: Determines if a graph has a path that visits every VERTEX exactly once.

Key Difference from Eulerian: - Eulerian = visit every **edge** once - Hamiltonian = visit every **vertex** once

Algorithm used: Backtracking — tries all possible vertex orderings. This problem is **NP-complete** (no known fast algorithm).

6. Complement Graph

What is a Complement Graph?

Given a graph $G = (V, E)$, its complement $\bar{G} = (V, \bar{E})$ is a graph that: - Has the **same vertices** as G - Has an edge between two vertices **if and only if** that edge does NOT exist in G

The Fundamental Invariant

$$|E(G)| + |E(\bar{G})| = n(n-1)/2$$

The total edges in G plus edges in $\bar{G}$ always equals the number of edges in the complete graph K_n .

How GraphLab Computes It

Algorithm (Edge Inversion — from [complementGraph.ts](#)):

1. **Index all existing edges** into a hash set for $O(1)$ lookup
2. **Enumerate all possible pairs** of vertices:
3. For undirected: check all pairs (i, j) where $i < j$
4. For directed: check all ordered pairs (i, j) where $i \neq j$
5. **If a pair has NO edge in G** , add it as an edge in $\bar{G}$
6. Compute the **degree inversion**: $\deg_{\bar{G}}(v) = (n - 1) - \deg_G(v)$

Self-Complementary Check: A graph $G \cong \bar{G}$ is only possible when: - $n \bmod 4 = 0$ OR $n \bmod 4 = 1$ - $|E(G)| = |E(\bar{G})| = n(n-1)/4$

Side-by-Side Visualization: GraphLab places the original graph G on the left and the complement $\bar{G}$ on the right, with the complement nodes labeled with primes ($A \rightarrow A'$).

Time Complexity: $O(V^2)$

Example

Original Graph G (Triangle K_3):

```
A - B
|   |
C ---
```

Complement $\bar{G}$:

```
A   B   C   (no edges -  $K_3$ 's complement is the empty graph)
```

Verification: $|E(G)| + |E(\bar{G})| = 3 + 0 = 3 = C(3,2) \checkmark$

7. Bipartite Graph Verification

What is a Bipartite Graph?

A graph is **bipartite** if its vertices can be divided into two disjoint sets X and Y such that every edge connects a vertex in X to a vertex in Y. Equivalently, a graph is bipartite if and only if it contains **no odd-length cycles**.

How GraphLab Checks It

Algorithm (BFS 2-Coloring — from [bipartite.ts](#)):

1. Pick any unvisited vertex, assign it **Color 0** (Set X)
2. Use **BFS** to explore neighbors:
3. For each neighbor: assign the **opposite color** (if current is 0, neighbor gets 1)
4. If a neighbor is **already colored** and has the **same color** as the current vertex → **NOT bipartite** (odd cycle found!)
5. If all vertices are colored without conflict → **bipartite**
6. If bipartite, GraphLab can **rearrange the layout** into a 2-column view (Set X on left, Set Y on right)

Time Complexity: $O(V + E)$

Example

Bipartite Graph:	Non-Bipartite Graph:
A — B	A — B
C — D	C — D
	\ /
	E
(4-cycle, even)	(has triangle = odd cycle)
✓ Bipartite	✗ NOT Bipartite

8. Havel-Hakimi Theorem

What it does

Determines whether a given sequence of integers can be the degree sequence of a simple graph (a "graphical sequence"). If yes, it constructs a realization.

The Algorithm

Havel-Hakimi Reduction (from [havelHakimi.ts](#)):

1. Given a sequence like [3, 3, 2, 2, 2]:
2. **Sort** the sequence in non-increasing order
3. Remove the **first element** d_1
4. **Subtract 1** from the next d_1 elements
5. **Repeat** until:
6. All elements are 0 → **graphical** ✓
7. A negative number appears → **not graphical** ✗

Step-by-Step Example

```
Input: [3, 3, 2, 2, 2]
```

```
Step 1: Sort → [3, 3, 2, 2, 2]
        Remove 3, subtract 1 from next 3 elements
        → [2, 1, 1, 2]
```

```
Step 2: Sort → [2, 2, 1, 1]
        Remove 2, subtract 1 from next 2 elements
        → [1, 0, 1]
```

```
Step 3: Sort → [1, 1, 0]
        Remove 1, subtract 1 from next 1 element
        → [0, 0]
```

```
All zeros → ✓ This is a valid degree sequence!
```

GraphLab can then **generate a graph realization** from this sequence and render it on the canvas.

9. Graph Isomorphism

What it does

Checks whether two graphs drawn on the canvas are **isomorphic** — structurally identical, meaning there exists a one-to-one vertex mapping that preserves all edges.

How GraphLab Checks It

Algorithm (from [isomorphism.ts](#)):

1. **Quick invariant checks** (if any fail → NOT isomorphic):
2. Same number of vertices?
3. Same number of edges?
4. Same degree sequence (sorted)?
5. **Vertex bijection search** using backtracking:
6. Try to map vertices of G_1 to G_2
7. At each step, check if the mapping preserves edges
8. If a complete valid mapping is found → **isomorphic** ✓
9. If no mapping works → **NOT isomorphic** ✗
10. **Connected component segmentation**: GraphLab automatically detects separate components on the canvas and treats the two largest as G_1 and G_2 .

Time Complexity: Worst case $O(V!)$ for the backtracking, but invariant checks prune most cases quickly.

10. Adjacency Matrix & Incidence Matrix

10.1 Adjacency Matrix

The **Adjacency Matrix** A is an $n \times n$ matrix where: - $A[i][j]$ = weight of edge from vertex i to vertex j - $A[i][j] = 0$ if no edge exists

For undirected graphs: The matrix is **symmetric** ($A[i][j] = A[j][i]$)

How GraphLab generates it (from [matrixGenerator.ts](#)): 1. Create an $n \times n$ matrix filled with zeros 2. For each edge (u, v) with weight w : - Set $A[u][v] = w$ - If undirected, also set $A[v][u] = w$

Example (undirected, unweighted):

Graph: A-B, B-C, A-C

Adjacency Matrix:

	A	B	C
A	[0]	[1]	[1]
B	[1]	[0]	[1]
C	[1]	[1]	[0]

10.2 Incidence Matrix

The **Incidence Matrix** B is an $n \times m$ matrix (n = vertices, m = edges) where: - **Undirected graph**: $B[i][k] = 1$ if vertex i is an endpoint of edge k , 0 otherwise - **Directed graph**: $B[i][k] = -1$ if edge k **leaves** vertex i , $+1$ if edge k **enters** vertex i

How GraphLab generates it: 1. Collect all unique edges and label them $e_1, e_2, e_3, \dots$ 2. Create an $n \times m$ matrix filled with zeros 3. For each edge e_k connecting vertices u and v : - If undirected: set $B[u][k] = 1$ and $B[v][k] = 1$ - If directed: set $B[u][k] = -1$ and $B[v][k] = +1$

Example (undirected):

Graph: A-B (e_1), B-C (e_2), A-C (e_3)

Incidence Matrix:

	e_1	e_2	e_3
A	[1]	[0]	[1]
B	[1]	[1]	[0]
C	[0]	[1]	[1]

11. AI Tutor & Image-to-Graph

AI Tutor

GraphLab includes an **AI Tutor** that can analyze the current graph and explain its properties. It uses: - **Offline heuristics engine**: Computes properties locally (Handshaking Lemma, degree analysis, connectivity, Euler conditions) without needing any API key - **Online LLM models** (optional): Connects to AI models (via OpenRouter, Groq, or Gemini) for deeper explanations

Image-to-Graph

Users can **upload a photo** of a hand-drawn or textbook graph, and AI will: 1. Parse the image to detect nodes and edges 2. Reconstruct the graph digitally on the canvas 3. Uses the Groq Vision API for image understanding

Summary of All Algorithms & Complexity

Feature	Algorithm	Time Complexity	Data Structure
BFS Traversal	Breadth-First Search	$O(V + E)$	Queue
DFS Traversal	Depth-First Search	$O(V + E)$	Stack
Dijkstra	Greedy + Relaxation	$O((V+E) \log V)$	Priority Queue
Bellman-Ford	Dynamic Programming	$O(V \times E)$	Distance Array
BFS Pathfinding	BFS + Path Reconstruction	$O(V + E)$	Queue
DFS Pathfinding	DFS + Path Reconstruction	$O(V + E)$	Stack
Prim's MST	Greedy + Min Edge	$O((V+E) \log V)$	Priority Queue
Eulerian Path	Hierholzer's Algorithm	$O(V + E)$	Stack
Hamiltonian Path	Backtracking	$O(V!)$	Recursion
Complement Graph	Edge Inversion	$O(V^2)$	Hash Set
Bipartite Check	BFS 2-Coloring	$O(V + E)$	Queue + Color Array
Havel-Hakimi	Greedy Reduction	$O(n^2 \log n)$	Sorted Array
Isomorphism	Invariant Check + Backtracking	$O(V!)$ worst	Hash Maps
Adjacency Matrix	Edge Enumeration	$O(V^2 + E)$	2D Array
Incidence Matrix	Edge Collection	$O(V \times E)$	2D Array

Note: This project is deployed live at <https://graph-lab-one.vercel.app/>